Don't Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks

Brian J Chan*
Chao-Ting Chen*
Jui-Hung Cheng*
Department of Computer Science
National Chengchi University
Taipei, Taiwan
{110703065,110703038,110703007}@nccu.edu.tw

Hen-Hsen Huang
Insititue of Information Science
Academia Sinica
Taipei, Taiwan
hhhuang@iis.sinica.edu.tw

Abstract

Retrieval-augmented generation (RAG) has gained traction as a powerful approach for enhancing language models by integrating external knowledge sources. However, RAG introduces challenges such as retrieval latency, potential errors in document selection, and increased system complexity. With the advent of large language models (LLMs) featuring significantly extended context windows, this paper proposes an alternative paradigm, cache-augmented generation (CAG) that bypasses real-time retrieval. Our method involves preloading all relevant resources, especially when the documents or knowledge for retrieval are of a limited and manageable size, into the LLM's extended context and caching its runtime parameters. During inference, the model utilizes these preloaded parameters to answer queries without additional retrieval steps. Comparative analyses reveal that CAG eliminates retrieval latency and minimizes retrieval errors while maintaining context relevance. Performance evaluations across multiple benchmarks highlight scenarios where long-context LLMs either outperform or complement traditional RAG pipelines. These findings suggest that, for certain applications, particularly those with a constrained knowledge base, CAG provide a streamlined and efficient alternative to RAG, achieving comparable or superior results with reduced complexity.

CCS Concepts

• **Computing methodologies** → **Discourse, dialogue and pragmatics**; **Natural language generation**; • **Information systems** → **Specialized information retrieval**.

Keywords

Large Language Models, Retrieval Augmented Generation, Retrieval-Free Question Answering

1 Introduction

The advent of retrieval-augmented generation (RAG) [1, 3] has significantly enhanced the capabilities of large language models (LLMs) by dynamically integrating external knowledge sources. RAG systems have proven effective in handling open-domain questions and specialized tasks, leveraging retrieval pipelines to provide contextually relevant answers. However, RAG is not without its drawbacks. The need for real-time retrieval introduces latency, while errors in selecting or ranking relevant documents can degrade the quality of the generated responses. Additionally, integrating retrieval and generation components increases system complexity, necessitating careful tuning and adding to the maintenance overhead.

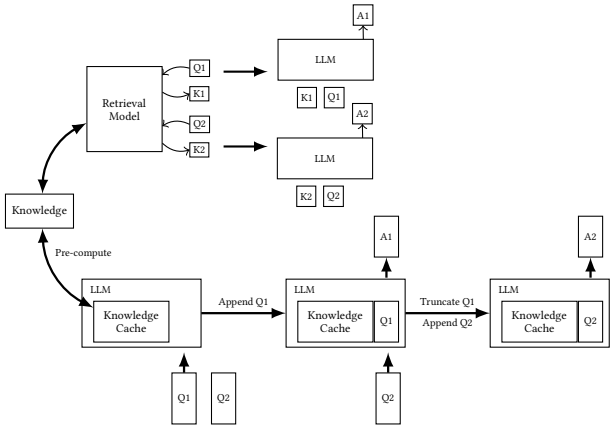

Figure 1: Comparison of Traditional RAG and our CAG Workflows: The upper section illustrates the RAG pipeline, including real-time retrieval and reference text input during inference, while the lower section depicts our CAG approach, which preloads the KV-cache, eliminating the retrieval step and reference text input at inference.

This paper proposes an alternative paradigm, cache-augmented generation (CAG), leveraging the capabilities of long-context LLMs to address these challenges. Instead of relying on a retrieval pipeline, as shown in Figure 1, our approach involves preloading the LLM with all relevant documents in advance and precomputing the key-value (KV) cache, which encapsulates the inference state of the LLM. The preloaded context enables the model to provide rich, contextually accurate answers without the need for additional retrieval during runtime. This approach eliminates retrieval latency, mitigates retrieval errors, and simplifies system architecture, all while maintaining high-quality responses by ensuring the model processes all relevant context holistically.

*Three authors contributed equally to this research.

Recent advances in long-context LLMs have extended their ability to process and reason over substantial textual inputs. By accommodating larger context windows, these models can assimilate extensive information in a single inference step, making them well-suited for tasks like document comprehension, multi-turn dialogue, and summarization of lengthy texts. This capability eliminates the dependency on real-time retrieval, as all necessary information can be preloaded into the model. These developments create opportunities to streamline workflows for knowledge-intensive tasks, potentially reducing or even eliminating the need for traditional RAG systems.

Recent studies [2, 4] have investigated the performance of long-context models in RAG tasks, revealing that state-of-the-art models like GPT-o1, GPT-4, and Claude 3.5 can effectively process large amounts of retrieved data, outperforming traditional systems in many scenarios. Findings suggest that as long as all documents fit within the extended context length, traditional RAG systems can be replaced by these long-context models. Similarly, Lu et al. [5] has demonstrated the benefits of precomputed KV caching to improve efficiency, albeit with the need for position ID rearrangement to enable proper functioning. Nonetheless, these methods remain vulnerable to retrieval failures inherent to RAG systems.

Through a series of experiments comparing traditional RAG workflows with our proposed approach, we identify scenarios where long-context LLMs outperform RAG in both efficiency and accuracy. By addressing the technical and practical implications, this paper aims to provide insights into when and why CAG may serve as a streamlined, effective alternative to RAG, particularly for cases where the documents or knowledge for retrieval are of limited, manageable size. Our findings challenge the default reliance on RAG for knowledge integration tasks, offering a simplified, robust solution to harness the growing capabilities of long-context LLMs. Our contributions are threefold as follows:

- Retrieval-Free Long-Context Paradigm: Introduced a novel approach leveraging long-context LLMs with preloaded documents and precomputed KV caches, eliminating retrieval latency, errors, and system complexity.
- Performance Comparison: Conducted extensive experiments showing scenarios where long-context LLMs outperform traditional RAG systems, especially with manageable knowledge bases.
- Practical Insights: Provided actionable insights into optimizing knowledge-intensive workflows, demonstrating the viability of retrieval-free methods for specific applications. Our CAG framework is released publicly.[1]

2 Methodology

Our CAG framework leverages the extended context capabilities of long-context LLMs to enable retrieval-free knowledge integration. By preloading external knowledge sources, such as a collection of documents $\mathcal{D} = \{d_1, d_2, \dots\}$, and precomputing the key-value (KV) cache C_{KV}, we address the computational challenges and inefficiencies inherent to real-time retrieval in traditional RAG systems. The operation of our framework is divided into three phases:

(1) **External Knowledge Preloading**
In this phase, a curated collection of documents $\mathcal{D}$ relevant to the target application is preprocessed and formatted to fit within the model's extended context window. The LLM $\mathcal{M}$, with parameters θ, processes $\mathcal{D}$, transforming it into a precomputed KV cache:

$$C_{\text{KV}} = \texttt{KV-Encode}(\mathcal{D}) \tag{1}$$

This KV cache, which encapsulates the inference state of the LLM, is stored on disk or in memory for future use. The computational cost of processing $\mathcal{D}$ is incurred only once, regardless of the number of subsequent queries.

(2) **Inference**
During inference, the precomputed KV cache C_{KV} is loaded alongside the user's query Q. The LLM utilizes this cached context to generate responses:

$$\mathcal{R} = \mathcal{M}(Q \mid C_{\text{KV}}) \tag{2}$$

By preloading the external knowledge, this phase eliminates retrieval latency and reduces risks of errors or omissions that arise from dynamic retrieval. The combined prompt $\mathcal{P} = \texttt{Concat}(\mathcal{D}, Q)$ ensures a unified understanding of both the external knowledge and the user query.

(3) **Cache Reset**
To maintain system performance across multiple inference sessions, the KV cache, stored in memory, can be reset efficiently. As the KV cache grows in an append-only manner with new tokens $t_1, t_2, \dots, t_k$ sequentially appended, resetting involves truncating these new tokens:

$$C_{\text{KV}}^{\text{reset}} = \texttt{Truncate}(C_{\text{KV}}, t_1, t_2, \dots, t_k) \tag{3}$$

This allows for rapid reinitialization without reloading the entire cache from disk, ensuring sustained speed and responsiveness.

The proposed methodology offers several significant advantages over traditional RAG systems:

- **Reduced Inference Time**: By eliminating the need for real-time retrieval, the inference process becomes faster and more efficient, enabling quicker responses to user queries.
- **Unified Context**: Preloading the entire knowledge collection into the LLM provides a holistic and coherent understanding of the documents, resulting in improved response quality and consistency across a wide range of tasks.
- **Simplified Architecture**: By removing the need to integrate retrievers and generators, the system becomes more streamlined, reducing complexity, improving maintainability, and lowering development overhead.

Looking forward, our approach is poised to become even more powerful with the anticipated advancements in LLMs. As future models continue to expand their context length, they will be able to process increasingly larger knowledge collections in a single inference step. Additionally, the improved ability of these models to extract and utilize relevant information from long contexts will further enhance their performance. These two trends will significantly extend the usability of our approach, enabling it to handle more complex and diverse applications. Consequently, our methodology is well-positioned to become a robust and versatile solution

[1] https://github.com/hhhuang/CAG

for knowledge-intensive tasks, leveraging the growing capabilities of next-generation LLMs.

Source	Size	# Docs	# Tokens	# QA Pairs
HotPotQA	Small	16	21k	1,392
	Medium	32	43k	1,056
	Large	64	85k	1,344
SQuAD	Small	3	21k	500
	Medium	4	32k	500
	Large	7	50k	500

Table 1: Overview of the SQuAD and HotPotQA test sets with varying reference text lengths, highlighting the number of documents, questions, and associated responses for each configuration.

3 Experiments

3.1 Experimental Setup

To evaluate the effectiveness of our proposed method, we conducted experiments using two widely recognized question-answering benchmarks: the Stanford Question Answering Dataset (SQuAD) 1.0 [6] and the HotPotQA dataset [7]. These datasets provide complementary challenges, with SQuAD focusing on precise, context-aware answers within single passages and HotPotQA emphasizing multi-hop reasoning across multiple documents. Each of both datasets consists of documents $\mathcal{D} = \{d_1, d_2, \dots\}$ paired with questions $Q_S = \{q_1, q_2, \dots\}$ and golden responses $\mathcal{R} = \{r_1, r_2, \dots\}$. These datasets provide a robust platform for assessing both single-context comprehension and complex multi-hop reasoning.

To investigate how different levels of reference text length impact retrieval difficulty, we created three test sets for each dataset, varying the size of the reference text. For example, in the HotPotQA-small configuration, we sampled 16 documents $\mathcal{D}_s \subset \mathcal{D}$ from the HotPotQA document set to form a long reference text. QA pairs associated with $\mathcal{D}_s$ were selected as test instances. The same methodology was applied to create test sets for SQuAD.

The dataset statistics are summarized in Table 1. As the number of documents (and hence the length of the reference text) increases, the task becomes more challenging, particularly for RAG systems. Longer reference texts increase the difficulty of accurately retrieving the correct information, which is crucial for LLMs to generate high-quality responses.

The primary task involves generating accurate and contextually relevant answers $\hat{\mathcal{R}} = \{\hat{r}_1, \hat{r}_2, \dots\}$ for the SQuAD and HotPotQA questions, based on the respective preloaded passages. By leveraging the precomputed key-value cache $C_{\text{KV}} = \texttt{KV-Encode}(\mathcal{D})$, our system generates responses $\hat{r}_i = \mathcal{M}(q_i \mid C_{\text{KV}})$ without relying on retrieval mechanisms during inference. This unified approach allows for direct performance comparisons against traditional RAG systems, highlighting the strengths and limitations of our method across diverse QA challenges.

The experiments were executed on Tesla V100 32G $\times$ 8 GPUs. For all experiments, we used the Llama 3.1 8B Instruction model as the underlying LLM across all systems, including both the RAG baselines and our proposed method. This model supports input sizes of up to 128k tokens, enabling the processing of extensive contexts. For our proposed method, the context of each dataset was preloaded into the model via a precomputed key-value (KV) cache. For SQuAD, the documents $\mathcal{D}_S$ were encoded into a KV cache $C^S_{\text{KV}} = \texttt{KV-Encode}(\mathcal{D}_S)$, while for HotPotQA, the documents $\mathcal{D}_H$ were encoded into $C^H_{\text{KV}} = \texttt{KV-Encode}(\mathcal{D}_H)$. These caches were stored offline and loaded during inference to eliminate the need for real-time retrieval, ensuring comprehensive access to all relevant information for each dataset.

3.2 Baseline Systems

The baseline RAG systems were implemented using the LlamaIndex framework,[2] employing two retrieval strategies: BM25 for sparse retrieval and OpenAI Indexes for dense retrieval. Each dataset—SQuAD and HotPotQA—was evaluated separately, with retrieval systems configured to fetch passages exclusively from the respective dataset to ensure focused and fair evaluation. The details of each baseline system are as follows:

(1) **Sparse Retrieval System (BM25)**: The first baseline system employed BM25 indexes for retrieval. BM25, a sparse retrieval algorithm, ranks documents based on term frequency-inverse document frequency (TF-IDF) and document length normalization. Given a query q_i, BM25 retrieves the top-k passages $\mathcal{P}_k = \{p_1, p_2, \dots, p_k\}$ from the indexed collection $\mathcal{D}$. These passages were then passed to the generator, $\mathcal{M}$, to synthesize answers:

$$\hat{r}_i = \mathcal{M}(q_i \mid \mathcal{P}_k) \tag{4}$$

BM25 provides a robust and interpretable retrieval mechanism, suited for tasks involving keyword matching.

(2) **Dense Retrieval System (OpenAI Indexes)** The second baseline utilized OpenAI indexes,[3] which employ dense embeddings to represent both documents and queries in a shared semantic space. For a query q_i, dense retrieval selects the top-k passages $\mathcal{P}_k$ that semantically align with the query, offering improved contextual understanding compared to sparse methods. These passages were similarly passed to the generator for answer synthesis as Equation 4. This system is particularly effective for questions requiring nuanced contextual matching beyond exact term overlap.

Our experiments were conducted on both the SQuAD and HotPotQA datasets to evaluate the performance of different systems in terms of similarity to ground-truth answers, measured using BERTScore [8]. For the RAG baselines, the top-1, top-3, top-5, and top-10 retrieved passages were used for inference. In contrast, our CAG utilized the preloaded context specific to each dataset to generate answers without retrieval constraints.

3.3 Results

As shown in Table 2, the experimental results revealed clear distinctions between our proposed method and traditional RAG systems. Our proposed approach achieved the highest BERTScore in most

[2]https://www.llamaindex.ai/framework
[3]https://cookbook.openai.com/examples/evaluation/evaluate_rag_with_llamaindex

Table 2: Experimental Results

Size	System	Top-k	HotPotQA BERT-Score	SQuAD BERT-Score
Small	Sparse RAG	1	0.0673	0.7469
		3	0.0673	0.7999
		5	0.7549	0.8022
		10	0.7461	0.8191
	Dense RAG	1	0.7079	0.6445
		3	0.7509	0.7304
		5	0.7414	0.7583
		10	0.7516	0.8035
	CAG (Ours)		**0.7759**	**0.8265**
Medium	Sparse RAG	1	0.6652	0.7036
		3	0.7619	0.7471
		5	0.7616	0.7467
		10	0.7238	0.7420
	Dense RAG	1	0.7135	0.6188
		3	0.7464	0.6869
		5	0.7278	0.7047
		10	0.7451	0.7350
	CAG (Ours)		**0.7696**	**0.7512**
Large	Sparse RAG	1	0.6567	0.7135
		3	0.7424	0.7510
		5	0.7495	0.7543
		10	0.7358	0.7548
	Dense RAG	1	0.6969	0.6057
		3	0.7426	0.6908
		5	0.7300	0.7169
		10	0.7398	0.7499
	CAG (Ours)		**0.7527**	**0.7640**

Table 3: Comparison of Generation Time

Dataset	Size	System	Generation Time (s)
HotpotQA	Small	CAG	0.85292
		w/o CAG	9.24734
	Medium	CAG	1.66132
		w/o CAG	28.81642
	Large	CAG	2.32667
		w/o CAG	94.34917
SQuAD	Small	CAG	1.06509
		w/o CAG	10.29533
	Medium	CAG	1.73114
		w/o CAG	13.35784
	Large	CAG	2.40577
		w/o CAG	31.08368

situations, outperforming both RAG systems. By preloading the entire context from the test set, our system eliminates retrieval errors and ensures holistic reasoning over all relevant information. This advantage is particularly evident in scenarios where RAG systems might retrieve incomplete or irrelevant passages, leading to suboptimal answer generation. These results underscore the robustness and efficiency of our method, especially for tasks requiring a unified understanding of the source material. While dense retrieval methods such as OpenAI Indexes perform better than sparse retrieval methods like BM25, both are inherently limited by their dependence on retrieval accuracy and ranking heuristics. Our approach bypasses these challenges, leveraging the long-context capabilities of the Llama 3.1 model to achieve superior performance.

Table 3 compares our CAG approach with standard in-context learning, where the reference text is provided dynamically during inference, requiring real-time KV-cache computation. The results demonstrate that CAG dramatically reduces generation time, particularly as the reference text length increases. This efficiency stems from preloading the KV-cache, which eliminates the need to process the reference text on the fly.

Moreover, CAG is also faster than traditional RAG systems, as it bypasses the retrieval stage entirely. Unlike RAG, CAG does not require retrieval or reference text input during inference, streamlining the process and further enhancing efficiency. These advantages make CAG an optimal solution for scenarios with extensive reference contexts, offering substantial time savings without compromising performance.

4 Conclusion

As long-context LLMs evolve, we present a compelling case for rethinking traditional RAG workflows. While our work emphasizes eliminating retrieval latency, there is potential for hybrid approaches that combine preloading with selective retrieval. For example, a system could preload a foundation context and use retrieval only to augment edge cases or highly specific queries. This would balance the efficiency of preloading with the flexibility of retrieval, making it suitable for scenarios where context completeness and adaptability are equally important.

References

[1] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).

[2] Quinn Leng, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin. 2024. Long Context RAG Performance of Large Language Models. *arXiv preprint arXiv:2411.03538* (2024).

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[4] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 881–893. https://doi.org/10.18653/v1/2024.emnlp-industry.66

[5] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2024. TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text. arXiv:2410.07590 [cs.CV] https://arxiv.org/abs/2410.07590

[6] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, Jian Su, Kevin Duh, and Xavier Carreras (Eds.). Association for Computational Linguistics, Austin, Texas, 2383–2392. https://doi.org/10.18653/v1/D16-1264

[7] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for

Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[8] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. [n. d.]. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.